General

*	all columns
AS "Column name"	column alias, AS is optional
DISTINCT	Only unique values, suppresses duplicates
table1 t1	Alias T1 for table1
table1 t1 JOIN table2 t2 USING (key)	Joins 2 tables (the PK column name is the same as the FK column name)
table1 t1 JOIN table2 t2 ON t1.FK = t2.PK (or ON t1.PK = t2.FK)	Joins 2 tables where the FK column value in one table must equal the PK column value in the other table
...LEFT [OUTER] JOIN…	all fields from the left table are shown
...RIGHT [OUTER] JOIN…	all fields from the right table are shown
...FULL [OUTER] JOIN…	all fields from both tables are shown
SELECT t1.column	selection from table(s)
FROM table1	Selects the table from which the columns come.
WHERE condition	filters the rows using the condition
GROUP BY	what the rows should be grouped by
HAVING condition	filters the rows after grouping using the condition
ORDER BY criteria1, criteria2, …	sorts (ascending) based on a column, column alias, column position, or an expression.
Criteria in ORDER BY [ASC|DESC] [NULLS FIRST|LAST]	ascending or descending, with empty values first or last.

Precedence

(…), * /, + -, ||, < > =, IS LIKE IN, BETWEEN, !=, NOT, AND, OR	order of operations

Comparison operators

<, >, <=, >=	less than, greater than, … or equal to
!= | <>	not equal to
=	is equal to
IS [NOT] NULL	is (not) equal to NULL
BETWEEN lower_bound AND upper_bound	between two values, including both values
IN (value1, value2,…)	is equal to one of the values between the parentheses
LIKE 'pattern'	matches a pattern, case-sensitive
% (in pattern)	any number of characters
_ (in pattern)	one arbitrary character

Logical operators

AND	requires both conditions to be true
NOT	requires the condition not to be true (including NOT IN, NOT BETWEEN, …)
OR	requires at least one condition to be true

Conversion functions

TO_CHAR(number, format)	converts a number to text
TO_CHAR(date, format)	converts a date to text
TO_DATE(string, format)	converts text to a date
TO_NUMBER(string, format)	converts text to a number

Format model for numbers

9	Placeholder for the position of a digit; no leading zeros if the number is too short (except after the decimal point)
0	Placeholder for the position of a digit; if there are not enough digits, the left side is padded with zeros.
.	Decimal point
,	Comma between thousands groups
FM	Suppresses spaces and zeros, both at the front and at the end (but not when 0 is used as a placeholder)
L	Symbol for the session's local currency
MI	Places a minus sign for negative numbers. It can be before or after the number; the minus sign is placed where MI appears.
S	Plus or minus sign for a number

Format model for dates

'text'	text, literally in the format
Day	day of the week in full (case-sensitive => DAY, Day, day)
DY	day of the week in three letters
D	Numeric day of the week (1 = Sunday, 7 = Saturday)
DD	numeric day of the month (01-31)
DDD	Numeric day of the year (001-366)
W	Numeric week of the month
WW	Numeric week of the year
th	After DD, adds the ordinal suffix st, nd, ….
MON	month, three letters (case-sensitive => MON, Mon, mon)
Month	month, written in full (case-sensitive => MONTH, Month, month)
MM	Numeric month, in 2 digits (01-12)
YY	year in 2 digits (current century)
RR	year in 2 digits
Year	year written out
YYYY	year in 4 digits
HH	Hour of the day (0-12)
HH12	Hour of the day (0-12)
HH24	Hour of the day (0-23)
MI	Minutes
SS	Seconds
MS	Milliseconds (000-999)
AM, am, PM, pm	Time indicator in the 12-hour system

Numbers

* + - /	arithmetic expressions
MOD(divisor, dividend)	modulo, the remainder after dividing the dividend by the divisor using integers
ROUND(number[, n])	rounds the number to n digits after the decimal point (no n means 0, n < 0 means n places before the decimal point).
TRUNC(number[, n])	truncates the number to n digits after the decimal point (no n means 0, n < 0 means n places before the decimal point).

Character functions

'this is text'	Text, case-sensitive
||	Concatenation between columns or with text.
INITCAP(text)	capitalizes the first letter of each word and makes the rest lowercase
LOWER(text)	converts text to lowercase
UPPER(text)	converts text to uppercase
CONCAT(c_1, c_2, …, c_n)	Concatenates text c1, c2 up to and including cn.
INSTR(text,subtext)	Returns the first position of the subtext in the text.
LENGTH(text)	returns the number of characters in the text
LPAD(c1, n, c2)	left-pads c1 to length n with characters from c2
REPLACE(text, s, r)	Replaces every occurrence of s in the text with r
RPAD(c1, n, c2)	right-pads c1 to length n with characters from c2
SUBSTR(text, start[, n])	part of the text starting at position start, n characters long (or until the end). The first character has position 1. Negative arguments count from the end.
TRIM(text1,text2)	removes the characters in text2 from the beginning and the end of text1
LPAD(text, number[,padding])	Pads the string on the left with padding (or spaces) up to number_count if the text contains too few characters. If the text is longer than number_length, the text is truncated on the right.
RPAD(text, number[,padding])	Pads the string on the right with padding (or spaces) up to number_count if the text contains too few characters. If the text is longer than number_length, the text is truncated on the right.
REPLACE(text, toReplace, replacement)	Replaces all toReplace character(s) in the text with the replacement character(s)

Date arithmetic

date1– date2	returns the number of days between two dates (type = numeric)
CURRENT_DATE	Current date (in the time zone of the current session)
CURRENT_TIMESTAMP	Current date and time (in the session time zone)
SYSDATE	Current date and time (in the database server's time zone)

MONTHS_BETWEEN(date1, date2)	Returns the months between date1 and date2
ADD_MONTHS(date, n)	Adds n months to the date
NEXT_DAY(date, 'weekday')	Returns the date of the weekday following the given date
LAST_DAY(date)	Returns the last day of the month
ROUND(date, unit)	Rounds the date using the unit 'month' or 'year'
TRUNC(date, unit)	Truncates the date using the unit 'month' or 'year'
EXTRACT(DAY FROM date)	Returns the numeric day of the month of the date.
EXTRACT(MONTH FROM date)	Returns the numeric month of the year of the date.
EXTRACT(YEAR FROM date)	Returns the 4-digit year of the date.

Conditional functions

NVL(a, b)	the first non-NULL argument is taken
COALESCE(a, b, c, …)	the first non-NULL argument is taken
NULLIF(expr1, expr2)	NULL if expr1=expr2, otherwise expr1

Conditional expression

CASE WHEN <conditional expression> THEN <return value> [WHEN <conditional expression> THEN <return value>] ELSE <return value> END	When the conditional expression is true, the corresponding return value is given.
The WHEN ….THEN … clause can be repeated as often as needed; if no conditional expression is true, the return value after ELSE is returned.	

Group functions

AVG(expr)	Average of all supplied non-NULL values
COUNT(expr)	number of rows (NULL values are not counted)
COUNT(*)	Number of supplied rows
MAX(expr)	maximum value of all non-NULL values
MIN(expr)	minimum value of all non-NULL values
SUM(expr)	Sum of all non-NULL values

Subquery

Retrieves values from 1 column (there may be multiple values).

Subquery returns 1 value:

Expression [=, <, >, >=, <=, !=] (subquery)	The expression is compared with the returned value based on the supplied operator.

Subquery returns multiple values:

Expression IN (subquery)	The expression value is equal to 1 of the values returned by the subquery
Expression NOT IN (subquery)	The expression value may not be equal to any of the values returned by the subquery.
Expression operator ANY (subquery)	The expression value matches, according to the operator used, one of the values returned by the subquery.
Expression operator ALL (subquery)	The expression value matches, according to the operator used, all of the values returned by the subquery.
=, <, >, >=, <=, !=	Possible operators to use with ANY or ALL.

DML

COMMIT;	Confirm change
ROLLBACK [TO savepoint_name] ;	Undo changes [up to …]
SAVEPOINT name;	Set an intermediate point in the transaction
INSERT INTO table[(column_name, column_name, …)] VALUES (value [, value, …)];	Add 1 row of data to a table
UPDATE table SET column_name = value [, column_name=value, …] [WHERE condition];	Modify data in a certain column in a certain table [in certain rows]
DELETE FROM table [WHERE condition];	Delete rows from a table

DDL

CREATE TABLE name (column_name datatype [DEFAULT expression] [CONSTRAINT name type …] [, column_name datatype …]);	Create a table with columns and the required constraints.
CREATE TABLE name (column_name datatype [DEFAULT expr] [column_constraint] , …. [, table_constraint] [, …]);	Constraints can be created at column level or at table level, with a specific syntax
column_name datatype CONSTRAINT constraint_name constraint_type	Column constraint (for PK, NOT NULL, UNIQUE)
, CONSTRAINT constraint_name constraint_type(column_name)	Table constraint (for PK and UNIQUE)
column_name datatype CONSTRAINT constraint_name REFERENCES table(PK)	Column constraint for FK

column_name datatype , CONSTRAINT constraint_name FOREIGN KEY(column_name) REFERENCES table(PK)	Table constraint for FK
column_name datatype CONSTRAINT constraint_name CHECK(condition)	Column constraint for check
column_name datatype , CONSTRAINT constraint_name CHECK(condition)	Table constraint for check
DROP TABLE [IF EXISTS] name[,…] [CASCADE|RESTRICT];	Delete 1 [or more] tables [with or without relationships/constraints]
ALTER TABLE name ADD type necessary_info	Add a column with datatype, a column-level constraint, …
ALTER TABLE name DROP type necessary_info	Delete a column, constraint, …
CREATE [OR REPLACE] VIEW name_vw [(column_name[, column_name …])] AS SELECT column[,column,…] FROM table [WHERE condition];	Create a view based on a subquery
DROP VIEW view_name_vw;	Delete a view
CREATE SEQUENCE sequence_name_seq [INCREMENT [BY] value] [MINVALUE value | NO MINVALUE] [MAXVALUE value | NO MAXVALUE] [START [WITH] value] [[NO] CACHE number] [[NO] CYCLE];	Create a sequence
NEXTVAL(sequence_name)	Retrieve the next sequence value
DROP SEQUENCE sequence_name;	Delete a sequence
CREATE INDEX name ON table(column);	Create an index on a column

PL/SQL: anonymous block

```
[DECLARE]
BEGIN
        -- statements
[EXCEPTION]
END;
```

PL/SQL: declarations

variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT expression];
variable_name table.column%TYPE;

CURSOR *cursor_name* [(*parameter_name datatype*, ...)] IS *select_statement*;
variable_name table or cursor%ROWTYPE;

PL/SQL: SELECT in PL/SQL

SELECT *select_list*
INTO {variable_name [, variable_name] …}
FROM table
…

PL/SQL: output in the console

DBMS_OUTPUT.PUT_LINE();

PL/SQL: exception handler

```
EXCEPTION
        WHEN exception1 [OR exception2…] THEN
                -- statements;
        [WHEN exception3 [OR exception4…] THEN
                -- statements;
        [WHEN OTHERS THEN
                -- statements;
```

Some predefined exceptions

NO_DATA_FOUND	ORA-01403
TOO_MANY_ROWS	ORA-01422
ZERO_DIVIDE	ORA-01476
VALUE-ERROR	ORA-06502
DUP_VAL_ON_INDEX	ORA-00001

Non-predefined exceptions: declare and associate them yourself

e_exception EXCEPTION;
PRAGMA EXCEPTION_INIT(e_exception, -oracle_error_code);

User-defined exceptions

Step1	declare	e_exception EXCEPTION;
Step2	raise	RAISE e_exception;
Step3	handle in EXCEPTION handler	

Exception-related functions

SQLERRM	Returns the error message associated with an error
SQLCODE	Returns the numeric value associated with an error
0	There is no exception
1	This is a user-defined exception
+100	NO_DATA_FOUND exception
Negative number	Another Oracle error

PL/SQL: Raise your own error and pass it to the application:

RAISE_APPLICATION_ERROR(error_no, error_message);

- Error no.: a number of your choice between -20 000 and -20 999

PL/SQL: procedure

```
CREATE [OR REPLACE] PROCEDURE procedure_name
        [(parameter1   [IN|OUT|IN OUT] datatype1,
          parameter2   [IN|OUT|IN OUT] datatype2, …)]
IS|AS
        [local variable declarations;]
BEGIN
        -- statements;
[EXCEPTION]
END [procedure_name];
```

PL/SQL: function

```
CREATE [OR REPLACE] FUNCTION function_name
        [(parameter1   [IN|OUT|IN OUT] datatype1,
          parameter2   [IN|OUT|IN OUT] datatype2, …)]
RETURN datatype
IS|AS
        [local variable declarations;]
BEGIN
        -- statements;
        RETURN expression;
[EXCEPTION]
END [function_name];
```

PL/SQL: trigger

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER}
{DELETE | INSERT | UPDATE [OF column [,column] ] } [ OR …]
ON table_name
[FOR EACH ROW [WHEN (condition) ] ]
[DECLARE
        local variable declarations;]
BEGIN
        -- statements;
[EXCEPTION]
END [trigger_name];
```

PL/SQL: IF

```
IF condition THEN
        -- statements
[ELSIF condition THEN
        -- statements]
[ELSE
        -- statements]
END IF;
```

PL/SQL: Basic loop

```
LOOP
```

```
        -- statements
        EXIT [WHEN condition];
END LOOP;
```

PL/SQL: WHILE loop

```
WHILE condition LOOP
        -- statements
END LOOP;
```

PL/SQL: FOR loop

```
FOR counter IN [REVERSE] start_value .. end_value LOOP
        -- statements
END LOOP;
```

PL/SQL: CURSOR

OPEN *cursor_name [(parameter_value,...)]*;
FETCH *cursor_name*;
CLOSE *cursor_name*;

PL/SQL: SQL cursor attributes

SQL%FOUND	Boolean, is TRUE if the last DML statement changed rows
SQL%NOTFOUND	Boolean, is TRUE if the last DML statement changed no rows
SQL%ROWCOUNT	Integer indicating how many rows were changed by the last DML statement